RAG 横纵分析报告

> 从"先搜再答"到"自主检索思考"，大模型如何学会查阅资料。

一、一句话定义

RAG（Retrieval-Augmented Generation，检索增强生成）是一种让大语言模型**在回答问题时先检索外部知识库，再基于检索到的资料生成答案**的技术范式。它的核心矛盾是：模型自身的参数化记忆（训练中学到的知识）有限且静态，而外部世界的信息是无限且动态的——RAG 是连接两者的桥梁。

> 🎯 **读完这篇你能**：设计一个基本的 RAG 系统（索引-检索-生成三段式流水线），理解 chunk 大小、embedding 模型选择、检索策略如何共同决定最终答案质量。

阅读指南

如果你只有 5 分钟：读一句话定义 + 横纵交汇洞察的 5.7（"第一性原理"追问）。你能在 5 分钟内理解 RAG 的本质矛盾和未来方向。

如果你想理解 RAG 但不碰技术细节：读技术背景的核心矛盾和标准流水线 + 纵向分析每个阶段的标题和开篇段 + 横向分析的产品矩阵总览。约 25 分钟。

如果你是 RAG 系统开发者/研究者：直接跳横向分析的检索策略对比和框架对比，横纵交汇洞察的 5.1–5.5（五个核心问题的回答）是精华。纵向分析的 3.3（基础设施成型：向量数据库与 RETRO）和 3.5（Agentic RAG 与 GraphRAG）是两个关键突破阶段。

二、技术背景

核心矛盾：模型的"记忆"终究不可靠

大语言模型有两个根深蒂固的缺陷，RAG 同时解决：

1. **知识静态性**：模型的知识冻结在训练数据截止的那一天。今天发生了什么新事、你公司内部有哪些文档、你的数据库里有什么最新记录——模型一概不知。
2. **幻觉**：当模型不知道答案时，它不会说"不知道"，而是会基于语言模式编造一个听起来合理的回答。

RAG 的解决方案简单直接：**不要靠模型记忆来回答，让它先去查资料。** 查到的资料既提供了最新信息，又约束了模型的回答范围——有凭有据，大幅降低幻觉。

RAG 的标准流水线

```
文档 → [分块] → [向量化] → [存入向量数据库]
                                          ↑
用户提问 → [查询处理] → [检索] → [重排序] → [LLM 生成] → 回答
```

四个核心环节：分块与索引、查询处理、检索与重排序、基于检索结果的生成。

> 前置知识请参考独立文档 《RAG_前置知识》，涵盖向量检索、嵌入模型、分块策略、混合搜索等基础概念的通俗讲解。

> 📚 **关联报告**

> – AI 搜索 — RAG 是 AI 搜索的核心技术栈，检索是第一步、生成是第二步

> – LLM 应用框架 — LangChain/LlamaIndex 等框架把 RAG 封装成开箱即用的能力

> – 大模型数据工程 — 检索质量的上限由分块、清洗、索引等数据工程决定

三、纵向分析：从概念诞生到主流范式

3.1 前 RAG 时代：检索和生成各行其道（2017–2019）

在 RAG 概念诞生之前，自然语言处理领域的两条线——信息检索和文本生成——几乎是完全独立发展的。

检索侧，**BM25** 是绝对霸主。这个基于词频和逆文档频率的排序函数从 1970 年代走来，以极其简单的逻辑统治了整个信息检索领域：词出现得越多、词越罕见，文档与查询的相关性越高。它简单粗暴但极端有效——即使在 2026 年，BM25 依然是混合检索中的关键组件，因为它在精确匹配（产品 ID、人名、专业术语）上无可替代。

生成侧，预训练语言模型正在爆发。BERT（2018）、GPT-2（2019）让"预训练+微调"成为范式，但它们只能生成文本，不能查询外部知识。

这个阶段的根本局限是：**检索和生成是两个独立系统，从未被设计为协同工作。** 做 QA 系统的工程师通常的做法是：用 BM25 检索候选段落，再单独训练一个 reader 模型从段落中提取答案——两端各自优化，缺乏端到端的信息流动。

3.2 概念诞生：REALM、DPR、RAG 三篇齐发（2020）

2020 年是 RAG 的"大爆炸"年份。三篇关键论文在同一年密集出现，从不同角度定义了检索增强生成的范式。

REALM（Google, 2020.02） 是第一篇提出"在预训练阶段就引入检索"的论文。Kelvin Guu 等人的关键洞察是：与其在模型训练完后外挂一个知识库，不如让模型在预训练阶段就学会"去查资料"。REALM 使用掩码语言模型（MLM）作为学习信号，在预训练的同时训练一个可微检索器，从 Wikipedia 中检索相关段落辅助预测被掩码的词。

REALM 在开放域 QA 上超越当时所有方法 4–16%，但它的代价极高——需要异步刷新整个知识库的索引来保证检索器和编码器的一致性。这种"训练时需要周期性重建索引"的特性，让 REALM 在实际部署中几乎不可用。但它的核心思想——**可微检索 + 外部知识**——为后来的工作奠定了方向。

DPR（Dense Passage Retrieval, Facebook, 2020.04） 直接改变了检索的技术路线。Karpukhin 等人抛弃了 BM25 的关键词匹配逻辑，提出了双编码器架构：用 BERT 分别编码查询和段落，通过向量内积计算语义相似度。

DPR 的历史价值在于：**它证明了语义检索可以取代关键词检索成为主流。** DPR 的 top–20 检索准确率比 BM25 高出 9–19%，而且向量化的段落可以预计算、预索引，查询时只需编码查询本身——这个特性让稠密检索在工程上变得可行。

RAG（Lewis et al., 2020.05） 是正式命名这个领域的关键论文。Patrick Lewis 和 Facebook AI 团队提出 RAG–Sequence 和 RAG–Token 两种范式，将预训练的 seq2seq 模型（BART）作为参数化记忆，与 Wikipedia 的稠密向量索引（DPR）结合。论文在开放域 QA 上全面超越当时 SOTA，更重要的是，它以清晰的命名和框架定义了一个新领域。

同年还有 **ColBERT**（2020.04）提出了"后期交互"机制——不用单个向量表示文档，而是保留每个 token 的词级向量，在检索时逐 token 比较。ColBERT 在精度和效率之间取得了巧妙平衡，成为重排序技术的核心基础。

这三篇论文的分工很有意思：**REALM 开了脑洞（在预训练中引入检索），DPR 造了轮子（双编码器稠密检索），RAG 做了产品（定义了完整的检索–生成框架）**。后来的所有 RAG 工作，或多或少都建立在这三个贡献之上。

3.3 基础设施成型：向量数据库与 RETRO（2021–2022）

RAG 概念提出后，行业面临的最大问题是工程落地。2021–2022 年，向量数据库的兴起解决了"存哪里"和"怎么搜得快"的问题。

Pinecone（2021）作为首个托管向量数据库 SaaS，让开发者通过几行 API 就可以完成向量存储和检索，无需自建 FAISS 集群。**Weaviate**（2021）、**Milvus**（2021）、**Qdrant**（2022）相继开源，形成了一个百花齐放的生态。这些工具让 RAG 从论文走向了生产环境。

同期，**RETRO（DeepMind, 2021.12）** 在模型侧做出了重要突破。Sebastian Borgeaud 等人的核心发现是：**检索可以大幅压缩模型参数需求**。RETRO 从 2 万亿 token 的数据库中检索文档片段，用 Chunked Cross–Attention 将检索信息注入生成过程。结果：7.5B 参数的 RETRO 在困惑度上匹配 175B 的 GPT–3——参数规模压缩了 23 倍。

这个结果隐含了一个深刻判断：**模型不需要记住一切，它只需要知道去哪里查。** 这个思想在 2026 年被广泛接受，但在 2021 年是反直觉的——当时的主流信仰是"参数越大越智能"。

HyDE（2022.12） 则提出了一个巧妙的概念：既然检索是在向量空间中找相似，而查询和文档的向量分布不同（查询通常简短抽象，文档具体详细），为什么不先生成一个"该有的文档长什么样"的假设，再用它去检索？HyDE 的"假设性文档嵌入"在零样本场景下显著提升了稠密检索的效果，思路延续了 Query2Doc（2023.03）的方向。

3.4 范式确立：三阶段体系（2023）

2023 年是 RAG 的"元年"——不是因为某一篇论文爆发，而是行业第一次系统性地理解了 RAG 的完整图景。

Gao 等人（2023.12） 的系统综述将 RAG 演划分为三个阶段，这个框架至今仍是行业共识：

阶段一：Naive RAG（朴素 RAG）。 最简单的"索引 → 检索 → 生成"线性管线。文档切分、向量化、存库；查询向量化、相似度搜索、取 top-k；拼接 prompt、生成回答。这条管线在今天看粗糙得像是原型，但在 2023 年，大量生产级应用就是靠这个跑起来的。

Naive RAG 的局限很快暴露：检索质量没有校验（搜到错的也照用）、分块策略太粗暴（固定大小切分经常把一句话切成两半）、查询太简短（用户的真实问题往往不适合直接用于检索）。

阶段二：Advanced RAG（高级 RAG）。 围绕 Naive RAG 的短板，行业在检索前和检索后两个方向加上了优化：

检索前：**查询重写**（LLM 把口语化的"它怎么样"改写为适合检索的精确查询）、**查询分解**（复杂问题拆成子问题分别检索）、**HyDE**（生成假设文档弥合查询-文档分布差异）。

检索后：**重排序**（Cross-encoder 对候选结果精确打分）、**混合检索**（向量搜索 + BM25 关键词搜索 + RRF 融合排序）、**分块策略演进**（语义分块、文档结构感知分块、句子窗口检索）。

阶段三：Modular RAG（模块化 RAG）。 不再满足于固定的线性管线，而是把 RAG 拆成可插拔模块——不同的任务场景可以组合不同的模块。Gao 等人（2024.07）正式提出了四种流程模式：线性（Linear）、条件（Conditional）、分支（Branching）、循环（Looping）。Modular RAG 的本质是：**RAG 不再是"一个流程"，而是"一个工具箱"**。

2023 年还有两个虽然没有 RAG 标签但影响深远的工作：

Self-RAG（2023.10） 让模型自我判断是否需要检索，并对检索结果的质量和生成答案的准确性进行反思。模型学会生成"反思 token"——是否需要检索、检索到的文档是否相关、生成的答案是否被支持。

"Lost in the Middle"（Liu et al., 2023.07） 揭示了长上下文的致命问题：信息放在开头和结尾时模型能准确提取，放在中间位置时性能急剧下降。这对 RAG 有深远影响——检索到的文档放在 prompt 的什么位置，直接影响模型能否利用它们。

当长上下文模型（Claude 100K, GPT-4 Turbo 128K）在 2023 年末陆续发布时，一个看似合理的问题出现了：既然模型能一次看 10 万 token，那还需要 RAG 吗？这个争论将贯穿 2024-2025 年。

3.5 "RAG is Dead" 争论与平息（2024-2025）

2024 年 2 月，Google 发布 Gemini 1.5 Pro——默认 1M token 上下文。社区的立即反应是："直接把整个知识库塞进 prompt 就行了，RAG 过时了。"

这个观点听起来很有道理，但它忽略了几个事实：

- **成本**：1M token 的 prompt，在 Claude Opus 4.6 上单次要 $15。做一次两次可以，做一百万次查询呢？
- **延迟**：处理 1M token 的首 token 延迟以秒计，而 RAG 的检索阶段只需几十毫秒。
- **精度**：Long Context 在单文档问答上确实比 RAG 好（+34%），但在跨文档综合上 RAG 反而更好（+67%）。

arXiv:2501.01880（2024.12） 系统对比了两者，核心发现是：Long Context 在 Wikipedia 类问答上通常优于 RAG，但 RAG 在对话类查询和泛化问题上更有优势。结论不是谁会取代谁，而是**两者适用于不同场景**。

到 2025-2026 年，争论已经平息。行业共识是 **"Context Engineering"**——把长上下文和 RAG 当作互补工具，根据任务特性混合使用。对单一长文档做深度分析（如分析一份 200 页的合同），长上下文直接出；对海量知识库做问答

（如从 100 万份文档中找到相关信息）或需要最新信息，RAG 是更好的选择。

3.6 结构化 RAG：GraphRAG 与 LightRAG（2024–2025）

传统 RAG 有一个隐蔽的弱点：它适合回答"某份文档里有什么"的问题，但不适合回答"这 1000 份文档综合起来最核心的主题是什么"这种需要全局综合的问题。

微软 GraphRAG（2024.04） 的解决方案是：不把文档切碎成片段，而是提取实体和关系构建知识图谱，然后通过社区检测算法（Leiden）生成层次化的社区摘要。查询全局问题时，汇总各个社区的摘要来回答。

GraphRAG 在回答的全面性和多样性上显著超越传统 RAG，但代价是索引成本极高——处理百万 token 级别的数据集，初始索引成本曾高达约 $33,000。

LightRAG（2024–2025） 是更轻量的替代方案：提取实体–关系图，但不做完整的社区摘要，而是用双层检索（低级实体 + 高级关系）来支持查询。成本降低 6000 倍，支持增量更新，适合动态数据场景。

3.7 Agentic RAG：从被动检索到自主推理（2024–2026）

RAG 的最新演进方向是**让模型不再是"被检索"的被动接收者，而是"主动规划检索策略"的决策者。**

CRAG（Corrective RAG, 2024.01） 在检索和生成之间加了一个"门卫"——轻量级评估模型给检索文档的质量打分。高质量的文档直接使用，低质量触发网络搜索补充，中等质量做分解重组去噪。

Agentic RAG 更进一步：Agent 自主决定何时检索、检索什么、要不要多轮检索、检索结果是否够用。典型工作流：接收问题 → 规划检索策略 → 执行检索 → 评估结果 → 如果不够再检索一轮 → 综合回答。这已经不是"检索+生成"，而是"规划+执行+推理"的循环。

2025–2026 年的最新进展中，**Reasoning Graphs（2026）** 将每个证据项的推理历史持久化为结构化边，在 4 跳问题上准确率提升 11 个百分点。**AutoSearch（2026）** 用强化学习自适应决定搜索步数。**GRIP（2026）** 将检索控制嵌入 token 解码过程，用 control–token 实现端到端的检索策略控制。

四、横向分析：竞争图谱

4.1 场景判断

RAG 是多层技术栈，包含向量数据库、RAG 框架、嵌入模型、重排序模型等不同层级。竞品充分，选取每层的代表性选手深入分析。

4.2 向量数据库对比

数据库	发布时间	部署方式	核心语言	最佳规模	云定价（参考）	开源
Pinecone	2021	托管	—	百万级	$70–100/月	❌
Milvus / Zilliz	2021	自托管/云	Go/C++	**十亿级**	$80–120/月	✅
Qdrant	2022	自托管/云	Rust	千万级	$65–85/月	✅
Weaviate	2021	自托管/云	Go	千万级	$65–95/月	✅
Chroma	2023	本地	Python	百万级以下	免费	✅
pgvector	2021	自托管	C	千万级以下	免费（Postgres）	✅

选型逻辑：

- **Pinecone**：零运维首选。Serverless 架构让没有专职 DevOps 的团队能快速上线。缺点是查询量大时成本不可预测，且无自托管选项——数据锁定风险存在。
- **Milvus**：大规模部署的工业标准。在 1 亿+ 向量规模下分布式扩展能力最强。代价是需要 Kubernetes + MinIO，运维复杂度高。
- **Qdrant**：性能敏感场景的最优选择。Rust 编写，在带复杂元数据过滤的搜索场景下性能是 Milvus 的 2–3 倍。单位性能成本最低。
- **Weaviate**：功能最丰富。内建向量化模块（不需要外部 embedding 服务即可直接处理原始数据）、原生混合搜索（BM25 + 向量）。适合内容密集的多模态 RAG 应用。
- **Chroma**：快速原型的神器。`pip install chromadb` 就能用，API 简洁。但不适合大规模生产。
- **pgvector**：如果你的团队已经在用 PostgreSQL，pgvector 可以让 RAG 零额外基础设施。适合千万级以内规模。

2026 年的一个趋势：向量数据库的炒作已经结束，用户开始意识到最重要的不是数据库本身，而是"数据管道的质量"。社区共识：75% 的 RAG 生产问题来自数据准备阶段（文档解析、清洗、分块），而非向量数据库选型。

4.3 RAG 框架对比

框架	发布时间	定位	最佳场景	核心优势	GitHub Stars
LangChain	2022.10	编排	复杂 Agent 工作流	700+ 集成，LangGraph	最高
LlamaIndex	2022.11	检索精度	企业知识库	高级索引（树/图），检索准确率高 35%	第二
Haystack	2022	生产 NLP	受监管行业（金融/医疗）	模块化可审计管道，内建评估	第三
Dify	2023	可视化开发	非技术团队快速原型	拖拽工作流	第四
Semantic Kernel	2023	企业 .NET	Azure/微软生态	M365 Copilot 治理集成	—

LangChain vs LlamaIndex 的分工格局：

2026 年，这两个框架形成了清晰的"组合式堆栈"：LlamaIndex 做高精度检索（树索引、关键词表、图索引），LangChain/LangGraph 做复杂编排（多步 Agent 工作流、有状态执行）。

LangChain 在 2024-2025 年经历了 API 变动频繁的争议，但 LangGraph 的加入巩固了它在 Agent 编排上的领先地位。LlamaIndex 则专注于"检索"这个核心场景，在处理复杂文档结构（PDF 中的嵌套表格、多层次标题）时的表现优于 LangChain。

Dify 是 2026 年最大的可视化 RAG 平台——非技术人员可以通过拖拽构建完整的 RAG 流程。

DSPy 走了一条不同的路：不写 prompt，用"编译器"范式自动优化 prompt。框架开销最低（~3.53ms），适合追求极致性能和自动化的团队。

4.4 嵌入模型与重排序

嵌入模型：

模型	发布方	维度	支持 MRL	多语言	MTEB 排名
text-embedding-3-large	OpenAI	3072	✅	✅	高
text-embedding-3-small	OpenAI	1536	✅	✅	中高
BGE-M3	BAAI	1024	❌	✅ (100+)	高
Cohere Embed v3	Cohere	1024	❌	✅ (100+)	中高
Qwen3 embedder	Alibaba	—	✅	✅	中高

MRL（Matryoshka Representation Learning） 是 2025-2026 年的关键进化——同一模型可以输出不同维度的嵌入，推理时按需截断（256→512→1024 维），在精度和效率之间灵活权衡。

重排序模型：

模型	延迟	精度	适用场景
BGE-reranker-v2	中	高	通用场景，首选
Cohere Rerank	中	高	商业 API，无需自托管
Qwen3-Reranker	中	高	中文场景，性能佳
monoBERT	中	中高	经典方案
LLM-as-judge	高	最高	对质量要求极高的最终过滤

生产推荐流水线：混合搜索取 Top 50-100 → Cross-encoder 重排序取 Top 5-10 → 送入 LLM。

4.5 GraphRAG vs 传统 RAG vs Agentic RAG

维度	传统 RAG	GraphRAG	Agentic RAG
检索粒度	文本片段	实体/关系/社区摘要	多轮、多源融合
全局推理	弱（局部片段）	强（层次化摘要）	中-强（多轮推理）
索引成本	低	极高	中（依赖工具）
增量更新	容易	困难	容易
最佳场景	事实问答	全局综合分析	多跳推理、复杂任务
代表	LangChain + 向量库	GraphRAG + LightRAG	Self-RAG, CRAG, AutoSearch

4.6 用户口碑与社区共识

Reddit r/RAG 和 r/LangChain 上反复出现的热门话题：

- "从 Pinecone 开始的团队都会遇到成本墙和供应商锁定问题。pgvector 或 Qdrant 是更好的长期选择。"
- "75% 的问题来自数据准备，不是模型选择。文档解析比向量数据库选择重要 10 倍。"
- "RAG 应该被当作工具调用（Tool Use）来使用——让 Agent 能够对检索结果进行推理，而不是简单的向量搜索。"
- "分块策略对最终准确率的影响（约 35%）超过重排序（约 28%）和上下文增强（约 22%）——花最多时间在分块上。"

Hacker News 上的高赞评论：

- "最大的转变是从'更好的分块+向量搜索'转向混合搜索+Agentic RAG 流水线，配合更强的重排序和查询重写。"
- "2026 年，约 67% 的生产级 LLM 部署使用了检索增强技术——RAG 已经从'新潮技术'变成了'基础设施'。"

4.7 RAG 评估基准

基准	发布	测试重点
RGB (RAG Benchmark)	2023	首个系统化 RAG 评估框架
CRUD-RAG	2025	覆盖创建/读取/更新/删除四种操作
RAGAS	2023	自动化评估框架（忠实度+答案相关度+上下文相关度）
DeepEval	2024	"RAG Triad"（答案相关、忠实、上下文精度）
LangSmith	2024	生产环境 RAG 流水线调试和监控

五、横纵交汇洞察

5.1 RAG 的"隐形胜利"：它让 LLM 在生产环境中变得可用

纵向历史中最显著的现象不是 RAG 本身的技术突破，而是 **RAG 让大模型从"演示品"变成了"产品"**。

2023 年初的 LLM 应用面临的核心困境是：模型在演示时无所不能，在生产环境中却频繁出错——幻觉、知识过期、不可控的输出。RAG 不是解决了所有问题，但它提供了一个关键的"兜底机制"：当模型不确定时，它至少可以去查资料。

2026 年，约 67% 的生产级 LLM 部署使用了检索增强技术。这个数字说明的不是 RAG 的技术优越性，而是它的**实用主义胜利**：与其花几年时间解决幻觉和知识更新的问题，不如几年内先让模型学会查资料。

5.2 "RAG is Dead" 争论的启示：技术变迁从来不是替代

2023-2024 年的"RAG is Dead"争论，暴露了行业的一个认知偏差：**人们倾向于认为新技术会完全取代旧技术。**

实际上，长上下文没有取代 RAG，RAG 也没有被长上下文取代。2026 年的共识是"Context Engineering"——把两者当作互补工具。长上下文适合单文档深度分析，RAG 适合海量知识库的精准检索——各自的优势正好是对方的短板。

这个 pattern 在 AI 史上反复出现：CNN 没有杀死 RNN，Transformer 没有杀死 CNN。**"替代"是罕见的，"共存"是常态。**

5.3 Naive → Advanced → Agentic：RAG 的螺旋升级

回头看 RAG 的三阶段演进，可以发现一个清晰的 pattern：**每次演进都在给 RAG 增加"智能"。**

- Naive RAG：没有智能，只有固定流程
- Advanced RAG：加入优化模块，但流程仍然是线性的
- Agentic RAG：让模型自己决定检索策略

这个 pattern 揭示了一个趋势：**RAG 正在从"外挂技能"变成"模型内生能力"。** Self-RAG 的"反思 token"、GRIP 的"control-token 检索控制"——这些工作都在把检索决策从外部框架转移到模型自身的推理过程中。

最终形态可能是：**模型不再需要"RAG"这个独立步骤。** 它可以在生成过程中自然地决定"这里我需要查一下资料"，执行查询，继续生成——就像人类在写作时翻开一本书查阅某个数据一样流畅。这个"RAG 内化"的趋势可能在未来 2-3 年重塑 LLM 的架构。

5.4 三原则：什么让 RAG 系统从"能用"到"好用"

从数据工程到生产部署，RAG 系统成功的三个关键原则：

1. **数据质量 > 模型选择**。社区经验反复证明：75% 的 RAG 问题来自数据准备阶段。文档解析、分块策略、元数据保持——这些"脏活"对系统质量的影响远超向量数据库选型或嵌入模型选择。
1. **混合搜索是标配，重排序必须做**。2026 年的生产级 RAG，混合搜索（稠密+稀疏）和 Cross-encoder 重排序不再是"可选的加分项"，而是"必选的基础配置"。单一向量检索的召回率和精度都不足以支撑生产环境。
1. **查询处理决定上限**。用户的原始查询很少适合直接用于检索（太短、含指代、太口语化）。查询重写、多查询扩展、查询分解——这些"检索前"的处理工作，往往决定了 RAG 系统性能的上限。

5.5 GraphRAG 的启示：结构化优于平面化

GraphRAG 的工作说明一个更深层的道理：**对于需要全局理解的任务，"平面化"的片段检索永远不够。**

传统 RAG 把文档切碎成一个个孤立的片段，丢失了文档间的关联和文档内的层级结构。GraphRAG 把结构加回来了——实体提取、关系构建、社区摘要——检索结果不再是"一段文字"，而是"一个有上下文关联的知识单元"。

LightRAG 证明了结构化的方向是对的，只是不需要 GraphRAG 那么极端的成本。**"适度结构化"可能是 RAG 的未来方向**——不是所有场景都需要知识图谱，但在需要全局推理的场景中，纯向量搜索是不够的。

5.6 三个未来剧本

剧本一：Agentic RAG 成为主流（概率 65%）

检索不再是"一次性的查资料"，而是 Agent 工作流中的一个子任务。模型自主规划检索策略、多轮检索、对检索结果进行推理。LangChain/LangGraph + LlamaIndex 的组合将成为标准技术栈。**最可能的路径——所有条件都已成熟，唯一缺的是 Agent 可靠性的进一步提升。**

剧本二：检索内化成模型原生能力（概率 25%）

模型不再需要外挂检索器，而是在架构层面内置检索能力。类似 RETRO 但更成熟的方案出现——模型在生成过程中自然地决定是否需要查资料，检索成为一个"原生操作"而非"调用外部工具"。如果某个厂商能在下一代架构中实现这一点，将颠覆当前的 RAG 框架生态。

剧本三：RAG 被上下文成本的急剧下降边缘化（概率 10%）

如果推理成本继续以当前速度下降（2023-2026 年约下降了 80%），再过 2-3 年，直接把整个知识库塞进 prompt 的成本可能低于运行一套 RAG 管线的维护成本。但即使在成本持平的情况下，RAG 在延迟（几十毫秒 vs 几秒的首 token 延迟）上的优势仍然不可替代。

5.7 RAG 的"第一性原理"追问

抛开所有技术细节，RAG 的终极问题非常简单：**我们应该让模型"记住"知识，还是"去查"知识?**

RETRO 的 7.5B 参数匹敌 GPT-3 175B 的困惑度，暗示了"查"的效率远高于"记"。

但另一方面，人类的"知识"从来不是单纯的"记住什么"和"去哪里查"的二分。专家能快速回答领域问题的原因，不是因为他们的记忆力好，而是因为他们理解的"深层结构"——专家不需要去"查"一张桌子有几条腿。这个"深层结构"的建立，需要大量的参数化学习。

所以 RAG 可能永远不会是"全部答案"。最强大的 AI 系统，最终可能是"记得足够多来建立深层理解，同时知道去哪里查最新和稀缺的信息"——**参数化记忆和检索记忆的融合**，而不是谁取代谁。

七、实战：面试题、场景题、应用题

面试题

Q1: RAG 和长上下文模型的本质区别是什么？各适合什么场景？

考察对不同技术路径的理解深度，而非复述定义。

核心区别：RAG 是"需要时去图书馆查资料"，长上下文模型是"把所有资料背下来再作答"。两者解决同一个问题（给模型注入外部知识），但选择完全相反的技术路线。

RAG 把知识存储在外部（向量数据库），查询时检索相关片段喂给模型。优点：知识更新零成本（改数据库即可），知识容量近乎无限（加服务器就行），检索来源可追溯。缺点：多了一整套工程复杂度（分块、索引、检索、重排序），检索失败则答案全错。

长上下文模型把知识直接放进 prompt（Gemini 2M context 能塞几本书）。优点：零工程复杂度（prompt 里贴就行了），模型能直接"看到"全部上下文而非依赖检索器的中间层。缺点：推理成本随 context 线性增长（每轮对话都重新处理所有 context），检索精度随位置衰减（Lost in the Middle 效应——模型对长文本中间位置的关注度显著低于开头和结尾）。

选型指南：

- 知识库 >10 万文档、频繁更新 → RAG
- 知识量小（<1 万 token）、一次性使用 → 长上下文模型
- 两者不互斥——"用 RAG 检索 + 把检索结果放进长上下文模型"是目前的最优解

Q2: 分块策略为什么重要？chunk_size 怎么选？

考察对 RAG 核心工程决策的理解。

分块策略为什么重要：RAG 的检索粒度由分块决定。块太小（100 token）→ 检索到的片段缺少上下文，模型看不懂"孤立的一句话"是什么意思。块太大（2000 token）→ 检索精度下降（包含太多无关信息），且容易超出模型的上下文窗口。

chunk_size 的权衡：

chunk_size	优点	缺点
小（100–256 token）	检索精度高、相关性判断准确	缺乏上下文，答案可能不完整
中（512–1024 token）	精度和上下文的平衡点	多数场景的最优选择
大（2048+ token）	上下文完整、一次检索覆盖更多信息	精度下降、噪声增多、成本更高

实际建议：对于大多数企业知识库场景，**512 token + 10–20% 重叠（overlap）**是经过大量实践验证的起步配置。重叠确保跨块边界的信息不丢失。具体值需要根据你的文档类型微调——法律合同偏大（条款不可拆分），FAQ 偏小（每条 QA 独立）。

Q3: 检索质量差的时候，从哪几个方向排查？

考察实际工程排错能力。

排查层次（从快到慢）：

1. **查询本身有问题吗？** 用户问的是什么？原始查询是否太短、太模糊？→ 加查询重写/扩展（HyDE、Query2Doc），把短查询扩展成更完整的表述再检索
2. **Embedding 模型匹配吗？** 通用 embedding 模型在你的垂直领域可能效果很差（法律、医疗、金融）。→ 换领域特化的 embedding 模型，或用你的领域数据做 fine-tune
3. **分块大小合适吗？** 块太小（检索到的都是碎片）或太大（检索到的都是整篇文档）→ 调整 chunk_size 和 overlap
4. **只用向量检索？** 纯语义检索在精确匹配（产品 ID、人名、法规编号）上不如关键词检索。→ 上混合检索（Hybrid Search）：向量检索 + BM25 关键词检索，用 RRF（倒数排名融合）合并结果
5. **重排序做了吗？** 初检索返回的 top-20 里，真正相关的可能只有 3–5 条。→ 加 Cohere Rerank / BGE-Reranker 做第二道过滤
6. **检索到的内容被模型忽略了吗？** Lost in the Middle 效应——关键信息在检索片段的中间位置，模型没注意到。→ 把最相关的片段放在 prompt 的开头或结尾

场景题

场景 1：为律所设计合同审查 RAG

> 一家律所要为律师构建一个合同审查 RAG 系统。知识库是过去十年积累的 5 万份合同（每份 20–50 页）、法院判例和法规。律师需要查询"类似条款在判例中怎么判的""这类条款的标准措辞是什么"。系统要求检索准确率极高——法律场景下错误信息的后果不可接受。

架构选择：

1. **分块策略**：合同按条款分块而非按 token 数分块。用文档结构（标题、条款编号）做边界，每个条款一个 chunk。条款内嵌的引用条款单独成块。元数据标注合同类型、签署日期、管辖法院。
2. **检索策略**：Hybrid Search（向量 + BM25）+ 两阶段重排序。BM25 确保法规编号、法律术语的精确匹配。第一阶段用 BGE-Reranker-v2 做粗排（top-100→top-20），第二阶段用法律领域 fine-tune 的模型做精排（top-

20→top-5）。

3. **引用溯源**：每个回答的每一句必须带引用——具体到合同编号、条款编号、判例编号。不允许模型做任何法律判断——只呈现相关先例和条款，由律师自己下结论。
4. **兜底策略**：检索相关性分数低于阈值时，明确回复"未找到足够相关的先例"，而不是勉强生成模糊答案。法律场景下，"不知道"比"猜了一个"好 100 倍。

场景 2：RAG 系统上线后用户反馈答案不对，怎么办？

> 你部署了一个企业知识库 RAG 系统，用户反馈"问它去年的销售数据，它给出了三年前的"。你的排查和改进方案是什么？

排查路径：

1. **检索了哪些 chunk？** 开启 RAG 系统的检索日志，看这个查询实际检索到的 top-K 文档是什么。很可能是：用户问"去年"，但知识库里"去年"出现在 2023 年的文档标题里，而向量检索把这篇文档排在前面。→ 根源：缺乏时间感知的检索策略。
2. **元数据过滤做了吗？** 很多 RAG 系统的检索只看语义相似度，不看文档的时间/类型/来源。→ 加元数据过滤：用户问销售数据 → 只检索"报告类型=销售报告"，按日期倒排，取最新的。
3. **用户的查询被正确改写了吗？** 用户说"去年"，系统应该把它转成"2025 年"。→ 加查询改写步骤：用 LLM 解析查询中的时间表达式、实体名、缩写，展开后再检索。
4. **长期方案**：建立 RAG 质量监控——定期采样检索日志，人工标注检索质量，跟踪 precision@5 和 recall@5 的变化趋势，检测模型/数据的漂移。

应用题

应用 1：从零设计一个公司内部知识库 RAG 系统

> 你们公司有 10 万份内部文档（Wiki、设计文档、会议纪要、代码文档、周报），分散在 Confluence、Google Drive、Notion、GitHub 四个平台。员工每天花 1-2 小时搜索信息。设计一个覆盖全部知识源的 RAG 系统。给出完整技术方案、各环节的工程决策理由、以及上线后的监控指标。

第一步：数据接入与清洗

四个数据源通过各自的 API 拉取（Confluence API、Google Drive API、Notion API、GitHub API）。增量同步而非全量——每天凌晨拉取过去 24 小时的更新，全量刷新每季度做一次。

清洗阶段做三件事：

- 去重：跨平台的同一份文档（同一份设计文档可能在 Confluence 和 Google Drive 各有一份）用内容哈希去重
- 格式标准化：Markdown/HTML/PDF 统一转为纯文本 + 保留结构标注
- 元数据抽取：文档标题、作者、创建/修改日期、来源平台、文档类型（Wiki/设计文档/周报/代码文档）、所属团队

第二步：分块与索引

混合分块策略——不同文档类型不同大小：

- Wiki/设计文档：512 token + 10% overlap
- 代码文档：按函数/类边界分块（AST 解析）
- 会议纪要/周报：按议程条目/日期分块

索引层：向量索引（text-embedding-3-large）+ 稀疏索引（BM25），两者都存。混合检索用 RRF 融合。

第三步：查询处理

用户输入 → 查询改写（用轻量 LLM 做一次改写：展开缩写、解析时间表达式、补充上下文）→ 多路检索（向量 + BM25 各取 top-50）→ RRF 融合 → 重排序（BGE-Reranker top-50→top-10）→ 送进 LLM 生成答案。

第四步：生成与引用

System prompt 设计：告诉模型"你是公司内部知识助手，回答基于以下检索到的文档片段。如果检索结果不足以回答问题，明确说'根据现有资料无法确定'，不要编造。"

每个回答附带引用列表——文档标题、来源平台、最后修改时间。用户点击引用可直接跳转到源文档。

第五步：权限控制（关键）

这是企业内部 RAG 最容易被忽视但最致命的问题——不同员工有权限看不同文档。方案：

- 检索时先过滤——在向量搜索的 metadata filter 里加入用户的权限组
- 不把文档内容索引到第三方 SaaS（向量数据库必须自建）
- 敏感文档（HR、财务）完全不入库，或者入库但只在查询时检查用户的 SSO 权限

第六步：上线后的监控指标

指标	定义	目标
检索 Precision@5	top-5 检索结果中相关文档占比	>85%
检索 Recall@5	所有相关文档中被检索到的比例	>80%
答案准确率	人工抽检答案为"可用"的比例	>90%
引用准确率	引用确实支持所声称内容的比例	>95%
拒答率	模型无法回答时明确拒绝的比例	>80%（目标是该拒绝时拒绝）
幻觉率	答案中信息不被检索文档支持的比例	<5%
日活用户	每天使用系统的员工数	基线 + 增长
平均查询耗时	从用户提问到返回答案的延迟	<3s

八、信息来源

核心论文与文章

论文/文章	来源	访问时间
REALM	arxiv.org/abs/2002.08909	2026-04-30
DPR	arxiv.org/abs/2004.04906	2026-04-30
ColBERT	arxiv.org/abs/2004.12832	2026-04-30
RAG (Lewis et al.)	arxiv.org/abs/2005.11401	2026-04-30
RETRO	arxiv.org/abs/2112.04426	2026-04-30
HyDE	arxiv.org/abs/2212.10496	2026-04-30
Query2Doc	arxiv.org/abs/2303.07678	2026-04-30
Lost in the Middle	arxiv.org/abs/2307.03172	2026-04-30
Self-RAG	arxiv.org/abs/2310.11511	2026-04-30
RAG Survey (三阶段)	arxiv.org/abs/2312.10997	2026-04-30
CRAG	arxiv.org/abs/2401.15884	2026-04-30
GraphRAG	arxiv.org/abs/2404.16130	2026-04-30
Modular RAG	arxiv.org/abs/2407.21059	2026-04-30
RAG vs Long Context	arxiv.org/abs/2501.01880	2026-04-30

产品与技术来源

来源	URL	访问时间
Wikipedia RAG 条目	wikipedia.org	2026-04-30
Pinecone 官方文档	pinecone.io	2026-04-30
Milvus 官方文档	milvus.io	2026-04-30
LlamaIndex 文档	llamaindex.ai	2026-04-30
LangChain 文档	langchain.com	2026-04-30
Reddit r/RAG	reddit.com/r/RAG	2026-04-30
Reddit r/LangChain	reddit.com/r/LangChain	2026-04-30

本文是横纵分析系列的第 12 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时–共时分析、社会科学纵向–横截面研究设计、以及竞争战略分析的核心思想。